

Jenkins Declarative Pipeline – Complete Hands-On Guide

This document provides a complete end-to-end hands-on guide for Jenkins Declarative Pipelines. It includes Jenkins installation, tool installation, environment setup, and step-by-step solutions for all Jenkins CI/CD assignments.

1. Jenkins Installation (Ubuntu / EC2)

Step 1: Update system

```
sudo apt update -y
```

Step 2: Install Java (Required for Jenkins)

```
sudo apt install openjdk-11-jdk -y
```

Step 3: Add Jenkins repository

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo tee /usr/share/keyrings/jenkins-keyring.asc  
echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] https://pkg.jenkins.io/debian-stable binary/ | sudo tee  
/etc/apt/sources.list.d/jenkins.list
```

Step 4: Install Jenkins

```
sudo apt update -y  
sudo apt install jenkins -y
```

Step 5: Start Jenkins

```
sudo systemctl start jenkins  
sudo systemctl enable jenkins
```

Step 6: Access Jenkins

```
http://:8080
```

Step 7: Get initial admin password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

2. Required Tool Installation

Install Git

```
sudo apt install git -y
```

Install Docker

```
sudo apt install docker.io -y  
sudo systemctl start docker  
sudo systemctl enable docker
```

Add Jenkins user to Docker group

```
sudo usermod -aG docker jenkins  
sudo systemctl restart jenkins
```

Install Node.js

```
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -  
sudo apt install nodejs -y
```

Install Maven

```
sudo apt install maven -y
```

3. Jenkins Global Tool Configuration

Go to Jenkins Dashboard → Manage Jenkins → Global Tool Configuration

Configure:

- Git
- NodeJS

- Maven
- Docker (if plugin installed)

Save the configuration.

4. Assignment 1 – Pipeline Basics

Jenkinsfile

```
pipeline {
  agent any
  stages {
    stage('System Info') {
      steps {
        sh 'hostname'
        sh 'uname -a'
        sh 'whoami'
      }
    }
  }
}
```

5. Assignment 2 – Git Checkout & Build

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps {
        git 'https://github.com/YOUR_GITHUB_REPO.git'
      }
    }
    stage('Build') {
      steps {
        sh 'npm install'
      }
    }
  }
  post {
    success { echo 'Build successful' }
    failure { echo 'Build failed' }
  }
}
```

6. Assignment 3 – Docker Build

```
pipeline {
  agent any
  environment {
    IMAGE_NAME = 'YOUR_DOCKERHUB_USER/demo-app'
  }
  stages {
    stage('Docker Build') {
      steps {
        sh 'docker build -t $IMAGE_NAME .'
      }
    }
  }
}
```

7. Assignment 4 – Secure Docker Push

```

pipeline {
  agent any
  environment {
    IMAGE_NAME = 'YOUR_DOCKERHUB_USER/demo-app'
  }
  stages {
    stage('Push Image') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'dockerhub-creds', usernameVariable: 'USER',
        passwordVariable: 'PASS')]) {
          sh 'echo $PASS | docker login -u $USER --password-stdin'
          sh 'docker push $IMAGE_NAME'
        }
      }
    }
  }
}

```

8. Assignment 5 – Deployment

```

pipeline {
  agent any
  environment {
    IMAGE_NAME = 'YOUR_DOCKERHUB_USER/demo-app'
  }
  stages {
    stage('Deploy') {
      steps {
        sh '''
        docker stop demo || true
        docker rm demo || true
        docker run -d --name demo -p 3000:3000 $IMAGE_NAME
        '''
      }
    }
  }
}

```

9. Assignment 6 – Conditional Deployment

```

pipeline {
  agent any
  stages {
    stage('Deploy to Prod') {
      when {
        branch 'main'
      }
      steps {
        echo 'Deploying only on main branch'
      }
    }
  }
}

```

10. Assignment 7 – Full CI/CD Pipeline

```

pipeline {
  agent any
  environment {
    IMAGE_NAME = 'YOUR_DOCKERHUB_USER/demo-app'
  }
  stages {

```

```

stage('Checkout') {
steps { git 'https://github.com/YOUR_GITHUB_REPO.git' }
}
stage('Test') {
steps { sh 'npm test || true' }
}
stage('Build') {
steps { sh 'docker build -t $IMAGE_NAME .' }
}
stage('Push') {
steps {
withCredentials([usernamePassword(credentialsId: 'dockerhub-creds', usernameVariable: 'USER',
passwordVariable: 'PASS')]) {
sh 'echo $PASS | docker login -u $USER --password-stdin'
sh 'docker push $IMAGE_NAME'
}
}
}
stage('Deploy') {
steps {
sh 'docker run -d -p 3000:3000 $IMAGE_NAME'
}
}
post {
always { echo 'Pipeline completed successfully' }
}
}

```

11. Cleanup & Best Practices

- Stop Jenkins when not in use
- Remove unused Docker images
- Rotate credentials
- Use least privilege IAM roles
- Always delete EC2 after practice